



MYTHIC

Mythic PPA Estimator Datasheet

June 2026

Document Version v0.4
SDK Version v26.05.0

1	Overview	3
2	Features	3
2.1	Latency Estimation	3
2.2	Power Estimation	4
2.3	Die Area Estimation.....	4
3	Block Diagram	4
4	Components	5
5	Chip Configuration Options	6
6	Estimator Implementation.....	6
6.1	Latency Estimation	6
6.1.1	Assumptions and Limitations	8
6.2	Power Estimation	9
6.2.1	Analog Power	9
6.2.2	Digital Power	9
6.2.3	Limitations	9
6.3	Die Area Estimation.....	9
7	Estimator Outputs	10
8	PPA Estimator Enhancements	13
8.1	Performance Estimation Updates	13
8.2	Analog Power Estimation Updates	13
8.3	Digital Power Estimation Updates	13

MYTHIC

- [1 Overview](#)
- [2 Features](#)
 - [2.1 Latency Estimation](#)
 - [2.2 Power Estimation](#)
 - [2.3 Die Area Estimation](#)
- [3 Block Diagram](#)
- [4 Components](#)
- [5 Chip Configuration Options](#)
- [6 Estimator Implementation](#)
 - [6.1 Latency Estimation](#)
 - [6.1.1 Assumptions and Limitations](#)
 - [6.2 Power Estimation](#)
 - [6.2.1 Analog Power](#)
 - [6.2.2 Digital Power](#)
 - [6.2.3 Limitations](#)
 - [6.3 Die Area Estimation](#)
- [7 Estimator Outputs](#)
- [8 PPA Estimator Enhancements](#)
 - [8.1 Performance Estimation Updates](#)
 - [8.2 Analog Power Estimation Updates](#)
 - [8.3 Digital Power Estimation Updates](#)

1 Overview

The Mythic PPA Estimator is built on a C++-based functional simulator originally developed by Mythic for internal use in the design and optimization of the GEN 2 silicon architecture and its associated compiler toolchain. Unlike a cycle-accurate RTL simulation, which models detailed microarchitectural behavior at the level of individual pipeline stages and signal transitions, a functional simulator operates at a higher level of abstraction. This enables significantly faster execution and makes it well suited for design-space exploration, performance tuning, and compiler integration. However, this abstraction also introduces inherent limitations in timing accuracy, power estimation, and microarchitectural fidelity.

2 Features

To support early-stage performance, power, and area (PPA) analysis, the Mythic PPA Estimator extends the functional simulator with the capabilities described below. The specific implementation details for each of these features are provided in later sections of this document.

2.1 Latency Estimation

The estimator predicts the end-to-end latency of a neural network by simulating execution on a single input tensor, providing a high-level view of overall performance. Because it is not cycle-accurate, the results should be considered approximate, with an expected deviation from RTL-based timing analysis.

The performance estimator derives latency from event traces and metrics produced by the functional simulator. It estimates total processing time by bounding the amount of digital computation that can be completed during the analog circuits' multiply-accumulate (MAC) execution period. Since the chip contains multiple instances of the Analog Compute Engine (ACE), timestep progression is synchronized across all ACEs. If the digital workload exceeds the available execution window, the simulator extends the duration of the timestep to ensure that digital computation completes before any ACE begins the next MAC operation.

MYTHIC

Because the estimator does not model microarchitectural details such as SRAM contention or interconnect congestion, its timing estimates provide a high-level performance view suitable for architectural exploration rather than detailed RTL correlation.

2.2 Power Estimation

Power estimation is based on analog circuit simulations and high-level activity factors. While this approach offers useful directional guidance, it does not capture the fine-grained switching activity or signal-level accuracy available from RTL or gate-level power simulations.

The estimator uses two primary inputs:

1. The neural network graph generated by the Mythic DNN Compiler.
2. Data transfer activity between hardware components, captured during functional simulation of the compiled network.

These inputs enable analysis of hardware utilization, including the number of operations executed on compute units, estimated SRAM accesses, and the total data movement across the interconnect.

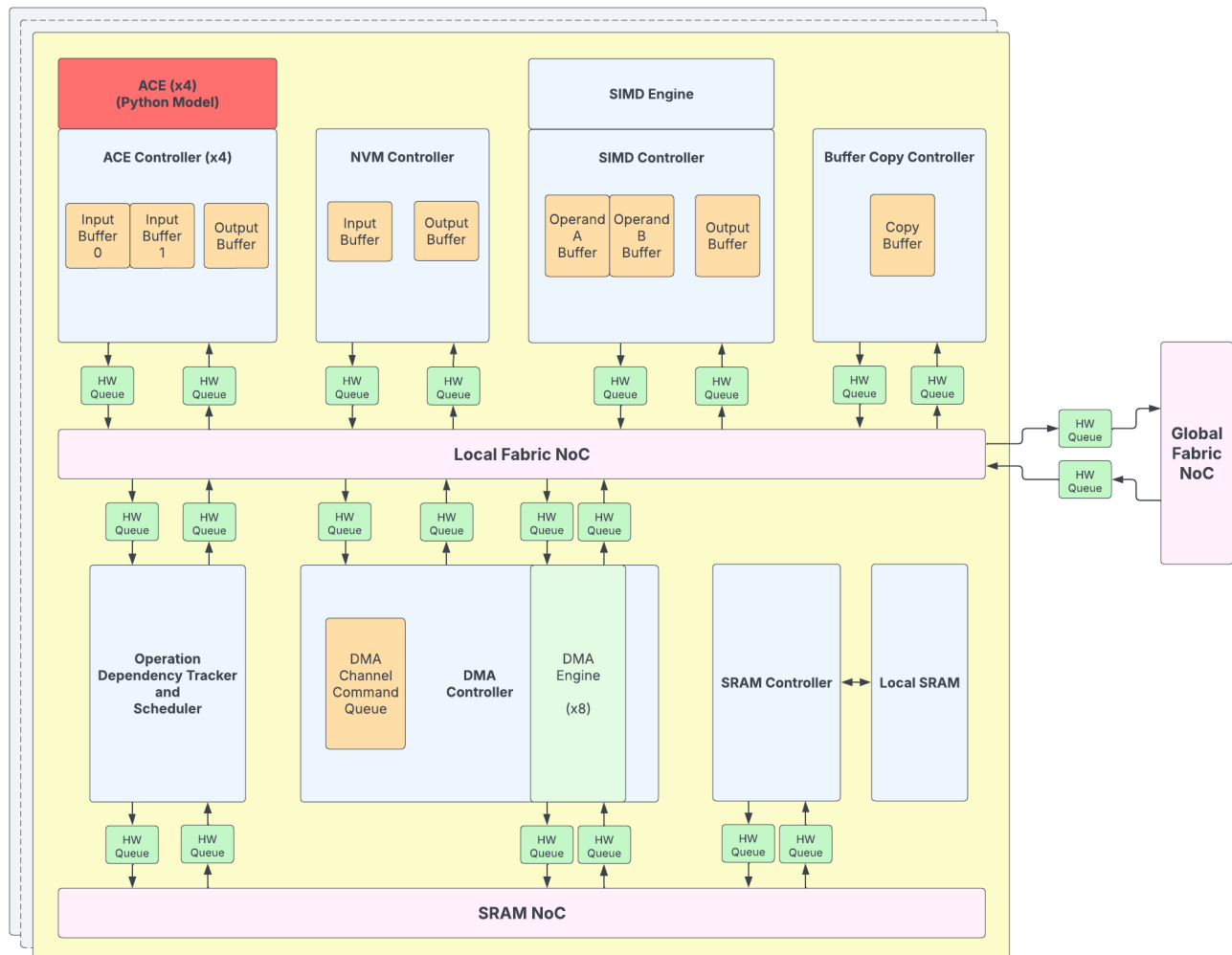
2.3 Die Area Estimation

Die area is estimated based on the number of Analog Compute Engines (ACEs) required to meet the model's parameter capacity and performance targets. This provides an approximate measure of the total silicon footprint. In a 3D-stacked chip configuration – where one die implements ACE circuits in 28nm and a second die implements digital logic in 5nm for execution control – the ACE die determines the overall silicon area.

3 Block Diagram

The figure below illustrates the major components implemented in the functional simulator and their interconnections. All primary hardware blocks shown at the top level of the diagram are modeled as functional modules written in C++.

Communication between blocks is represented by hardware queues abstracted in C++. Unlike an RTL implementation, where data transfers occur over pipelined interconnects and consume simulation time, the functional model performs queue operations instantaneously – blocks can push and pop elements in zero time. Each hardware queue is typically managed by an independent C++ thread, enabling concurrent execution and allowing multiple hardware blocks to process operations in parallel.



4 Components

The Mythic DNN Compiler maps the target neural network graph onto the functional simulator's hardware model. The functional simulator then executes the compiler-generated DNN program on a high-level functional representation of the hardware. This model captures the dataflow behavior and execution semantics of the architecture with sufficient accuracy to ensure functional correctness and verify that operations are scheduled and executed in the proper sequence.

Each simulated hardware component is multi-threaded, allowing independent operations to execute as soon as their input dependencies are satisfied. This enables the simulator to reflect the inherent parallelism of the underlying dataflow architecture. The block diagram above illustrates the primary hardware components modeled in the simulator, along with the key data and control buffers that influence scheduling and data movement. The simulator operates on a transaction-based execution model, where both data and control objects are transferred between components through discrete transactions.

To support parallel execution, each component runs in its own software thread. Threads enqueue outgoing transactions to local queues that are unique to each interface, and proceed with further computation. Similarly,

incoming transactions are dequeued and processed as input data becomes available. Inter-component communication is handled by a Network-on-Chip (NoC) model, which routes transactions between components via input and output queues. The NoC uses the same memory-mapped addressing scheme implemented in the actual hardware.

As an example, the four controller types shown at the top of the block diagram – ACE Controller, NVM Controller, SIMD Controller, and Copy Controller – each execute in separate software threads. These threads emulate hardware behaviors such as:

- Processing incoming read/write requests from other components via the NoC
- Executing compute or control operations
- Generating and dispatching control transactions to other components
- Managing scheduling-related functions such as credit tracking and dependency checking

This transaction-driven, multi-threaded simulation framework enables accurate modeling of the hardware's concurrent dataflow execution and provides valuable insight into system-level performance and behavior.

5 Chip Configuration Options

The functional simulator represents chip configurations through fixed virtual platforms. Each virtual platform specifies a particular chip configuration, including details such as the number of Analog Compute Engines (ACEs) – organized into groups of four called ACE Tiles – and hardware parameters like the total on-chip local SRAM capacity per ACE Tile.

Currently, the simulator supports a discrete set of virtual platforms, including:

1. 24Ace6Tile: 24 ACEs arranged in 6 ACE Tiles
2. 48Ace12Tile: 48 ACEs arranged in 12 ACE Tiles

Before execution in the functional simulator, the Mythic DNN Compiler must generate an executable binary of the neural network targeting the appropriate virtual platform. To ensure consistency between the compiler and the functional simulator, the PPA Estimator user interface does not allow users to manually specify the virtual platform. Instead, this information is automatically extracted from the compiler output and passed directly to the functional simulator.

Future releases will extend configuration flexibility to include additional parameters such as local SRAM capacity per ACE Tile and other architectural options.

6 Estimator Implementation

6.1 Latency Estimation

The functional simulator does not inherently model time in its execution of operations or data transfers. Instead, it defines a notional time window based on the execution of Analog Compute Engine (ACE) calculations, synchronizing all simulated activity to this reference.

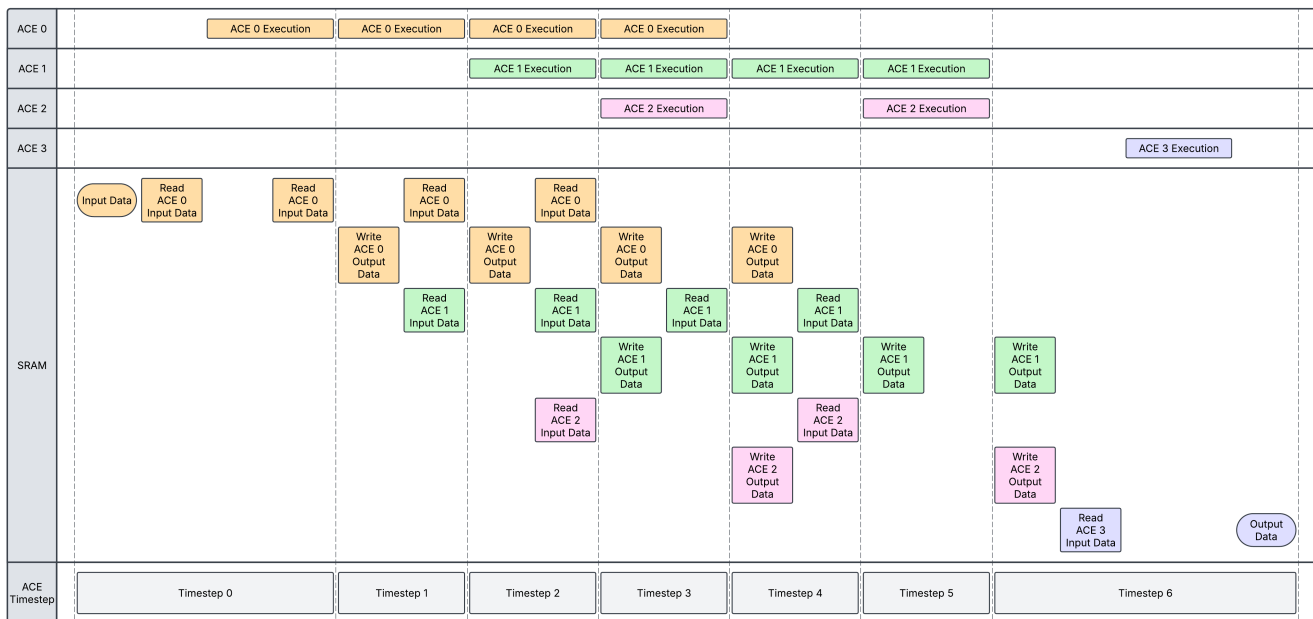
A key design principle of the Mythic dataflow architecture is that the digital logic should sustain continuous ACE operation at maximum throughput for the given DNN program. In other words, the ACEs should ideally execute back-to-back calculation operations without idle cycles or delay. When this condition is met, the total latency of the DNN program is determined by the cumulative time required to perform ACE operations along the longest dependency path in the network graph.

To model this behavior, the functional simulator groups all ACE calculations that are ready to execute into a timestep. During each timestep, the simulator executes all ready ACE operations, then advances the timestep

counter. This approach ensures that the number of executed timesteps corresponds closely to the theoretical minimum execution time for the DNN program.

An ACE calculation (or “ACE operation”) takes a single input vector of data and performs a multiply-and-accumulate (MAC) operation across a specified number of programmed “weight” vectors to generate an output vector of length equal to the specified number of weight vectors. Each ACE requires a fixed duration to complete a single ACE operation, regardless of the number of inputs or outputs. Based on this fact, the default duration of each timestep is equal to the fixed ACE calculation execution time. Therefore, the minimum total estimated latency for processing a frame through the DNN graph can be computed as the number of timesteps multiplied by the ACE operation time. Some timesteps may be extended to allow for activity that is estimated to take longer than the ACE calculation time.

This method necessarily involves broad assumptions and simplifications, some of which are discussed in more detail below.



In practice, the dataflow architecture may not always sustain ideal ACE utilization as described above. The Mythic architecture is also optimized for minimal power consumption and silicon area, which necessitates certain design tradeoffs. These tradeoffs can limit the digital logic’s ability to keep the ACEs operating at maximum throughput across all scenarios. The functional simulator accounts for these effects through several mechanisms.

For instance, the simulator monitors the volume of SRAM data read and written by each SRAM Controller during every timestep. If the total data movement exceeds the SRAM Controller’s designed throughput for that timestep, the simulator extends the timestep duration to accommodate the additional memory accesses. A similar adjustment is applied to other operations that may take longer than a single ACE calculation – such as extended SIMD Engine computations – to ensure that overall timing reflects these hardware resource constraints.

The reported performance estimate is based on the following formula:

```
Estimated Latency =  
max(  
    N_timesteps * t_ACE,  
    max_over_tiles(t_SRAM, read+write),  
    max_over_tiles(t_SIMD)  
)
```

where:

- `N_timesteps` – total number of ACE timesteps executed by the simulator
- `t_ACE` – fixed duration of a single ACE calculation operation
- `t_SRAM, read+write` – effective SRAM read+write duration per ACE Tile
- `t_SIMD` – effective SIMD Engine computation duration per ACE Tile

Each of the latter three terms is taken as the maximum observed duration across all ACE Tiles in the simulated system.

The first term, `N_timesteps * t_ACE`, corresponds to the idealized case where the digital subsystem maintains continuous ACE operation without stalls or throughput limitations. Under these conditions, the estimated latency closely approximates the actual execution time of an optimally utilized system.

The additional terms capture the impact of digital bottlenecks arising from bandwidth limits, memory contention, and extended computational workloads. By taking the maximum of these values, the estimator incorporates key system-level constraints while maintaining a simple, interpretable model.

6.1.1 Assumptions and Limitations

Because the performance estimator is derived from a functional simulator rather than a cycle-accurate or RTL-level model, several simplifying assumptions are applied to enable fast and scalable analysis. These assumptions are designed to provide high-level performance insight rather than precise timing correlation.

- **Uniform ACE Operation Time:** All ACE operations are assumed to complete in a fixed duration. In hardware, some variation may occur – for example, when portions of the ACE enter sleep mode to conserve power and require additional wake-up time before computation can begin.
- **No Scheduling Overhead:** The estimator does not model latency introduced by the scheduling system, assuming that scheduling overhead is negligible compared to compute latency.
- **Idealized Data Transfers:** Data movement between blocks or tiles is modeled as instantaneous, with no additional latency for transfer or routing.
- **Simplified SIMD Engine Timing:** SIMD Engine duration is estimated as a fixed function of the input data size. The model does not yet account for operations that may require more than one clock cycle per vector slice. Additionally, SIMD computations are assumed to complete entirely within the current timestep, regardless of output size or inter-timestep dependencies. This can lead to conservative (i.e., longer) timestep durations in some cases.
- **SRAM Contention:** Contention effects from SRAM bank collisions are represented by a fixed derating constant rather than a dynamically modeled interaction.

These abstractions enable the simulator to deliver fast, high-level latency estimates suitable for architectural exploration, configuration tuning, and hardware–software co-design.

6.2 Power Estimation

The power estimation framework combines both analog and digital power models to provide an overall energy profile for each virtual platform configuration (e.g., 24Ace6Tile, 48Ace12Tile).

6.2.1 Analog Power

Analog power is estimated using energy data derived from circuit-level SPICE simulations for the primary components of the Analog Compute Engine (ACE). The estimator performs a coarse-grained activity analysis, identifying which input AIDACs and output ADCs are active or idle during neural network graph execution. This enables differentiation between active and sleep power states, ensuring that both dynamic and static analog energy contributions are represented in the total estimate.

6.2.2 Digital Power

Digital power estimation for the analog NPU relies on two primary inputs:

1. The DNN Compiler-optimized neural network graph, which maps each layer or operation of the model to specific hardware components within the selected virtual platform configuration.
2. The data transfer activity between and within ACE Tiles, capturing both intra-tile and inter-tile communication patterns.

From the optimized graph, the estimator determines how individual neural network operations correspond to hardware units. For example, a *Conv* layer may map to an ACE Controller + ACE pair, while a *MaxPool* operation may map to a SIMD Controller + SIMD Engine. The graph also specifies the operation counts, loop iterations, and SRAM read/write volumes for each node. These metrics are used to accumulate energy contributions from each active hardware controller, local SRAM access, and associated control overhead.

The data transfer information further allows the estimator to account for the Network-on-Chip (NoC) energy required to move data between components and across tiles. Together, these factors capture the dominant contributors to digital power consumption.

6.2.3 Limitations

At present, the estimator excludes certain elements such as leakage power, clock tree power, chip I/O power, and interconnect distance effects. These will be incorporated in future versions once a mature RTL implementation is available.

6.3 Die Area Estimation

The area estimation methodology is based on the chip's hybrid 3D wafer stacking technology, which combines two process nodes: an analog die fabricated in 28nm and a digital die fabricated in 5nm. This integration enables high-density digital control and dataflow management while preserving the efficiency of analog computation.

The dataflow architecture is implemented on the digital die and manages the execution of Multiply-Accumulate (MAC) operations performed in the analog domain. The Mythic DNN Compiler ensures that the mapping of neural network parameters to the Analog Compute Engines (ACEs) meets performance goals (e.g., target frames per second) by effectively distributing computation across available resources within a given virtual platform.

Because the digital die is fabricated in the smaller 5nm process node, it provides sufficient bandwidth to satisfy the ACE input and output requirements through the vertical interconnects of the 3D stack. Consequently, the

overall die footprint is primarily determined by the analog die area, which scales with the selected virtual platform configuration.

Area estimates are therefore expressed in terms of the supported virtual platforms – for example, 24Ace6Tile or 48Ace12Tile – each corresponding to a specific number of ACE Tiles (groups of four ACEs) within the overall chip architecture.

7 Estimator Outputs

The table below details the PPA Estimator outputs, their definitions, and the methods used to compute them.

Item	Description
Analog NPU Performance Estimation Output	
Analog NPU Total Estimated Processing Time for Compiled ONNX File	Estimated processing time generated by the performance estimator using the ACE-based timestep estimation method, as described in Latency Estimation . Note that latency for the digital layers at the beginning and end of the ONNX file are accounted for separately as described below.
Analog NPU Estimated Frame Rate for Compiled ONNX File	Frame rate estimated by converting the total processing time into frames per second (fps).
Critical Path ACE Latency	Total estimated processing time accounting for only ACE operations (as if there were no additional latency due to digital operations).
Maximum SRAM Read/Write Time (over all ACE Tiles)	Maximum accumulated time spent reading and writing SRAM, with the maximum taken across all tiles.
Maximum SIMD Operation Time (over all ACE Tiles)	Maximum accumulated time spent on SIMD operations, with the maximum taken across all tiles.
Average SRAM Read/Write Time (averaged over all ACE Tiles)	Average accumulated time spent reading and writing SRAM, with the average taken across all tiles.
Average SIMD Operation Time (averaged over all ACE Tiles)	Average accumulated time spent on SIMD operations, with the average taken across all tiles.
Total Executed ACE Operations	Total number of ACE operations executed across all ACEs in the target chip configuration during simulation. An ACE operation is the calculation of an output vector from the input vector and programmed “weight” vectors as described earlier.

Item	Description
Total Executed ACE MACs	Total number of MAC operations executed across all ACEs in the target chip configuration during simulation. This value is derived from the compiled ONNX file and reflects applied parallelization and optimization strategies, so it may differ from the original ONNX file prior to compilation.
Maximum Theoretical ACE Execution Time (With No Parallelization Using 1 ACE)	Theoretical upper bound on execution time assuming the entire ONNX file is executed sequentially on a single ACE, with no parallelization applied.
Minimum Theoretical ACE Execution Time (With Even Parallelization Across 24 ACEs)	Theoretical lower bound on execution time assuming perfect parallelization of the ONNX file across all 24 ACEs with no system-level bottlenecks. This represents the minimum possible processing time, though it is not achievable in practice.
ACE Utilization (Minimum Theoretical Time/ Estimated Processing Time)	ACE utilization expressed as the ratio between the minimum theoretical processing time (assuming perfect parallelization with no system bottlenecks) and the ACE-based timestep estimated processing time produced by the performance estimator.
Total Data Bytes Read Across Chip	Total number of bytes read from SRAM, aggregated across all SRAM instances in the target chip configuration. This includes reads for both control flow and data buffers.
Total Data Bytes Written Across Chip	Total number of bytes written to SRAM, aggregated across all SRAM instances in the target chip configuration. This includes writes for both control flow and data buffers.
Estimated Die Area to Achieve Estimated Processing Time	Estimated die area based on the number of ACE Tiles required to meet network parameter capacity and system-level performance targets, as described in Die Area Estimation .
Digital NPU Performance Estimation Output	
Total Estimated MACs Targeting Digital NPU	Total number of MAC operations executed on Videantis digital compute (v-MP cores) units at the beginning and end of the network graph.
Digital NPU MAC Utilization	Utilization of available Videantis digital compute.

Item	Description
Digital NPU Cores	Total number of Videantis v-MP cores used for processing digital portion of the ONNX file.
Digital NPU Frequency	Clock frequency of the Videantis digital compute used for performance estimation.
Digital Estimated Frame Processing Time	Estimated processing time for the digital portion of the ONNX file.
Digital Estimated Frame Rate	Estimated frame rate for the digital portion of the ONNX file.
Combined NPU Performance Estimation Output	
Combined Analog + Digital NPU Latency	Combined (total) latency of the analog NPU and digital NPU processing the compiled ONNX file.
Combined Analog + Digital NPU Total Estimated Frame Rate	Combined (total) frame rate estimated by converting the combined processing time into frames per second (fps).
Power Estimation Output	
Process Nodes	Analog and digital process nodes used during power estimation (e.g., 28nm and 5nm).
Number of ACEs	Total number of ACEs used during power estimation (e.g., 24 ACES for the M2024 chip configuration).
Number of Digital NPU Cores	Total number of Videantis v-MP cores used for processing digital portion of the ONNX file.
Inference Rate	Inference rate (in frames per second) used during power estimation.
Analog NPU Estimated Power	Total estimated power of the analog NPU for the target chip configuration when running the compiled ONNX file. This estimate includes both analog and digital power, where the digital component covers all functional units and interconnect but excludes leakage, clock tree, and chip I/O power. The power estimation method is described in Power Estimation .

Item	Description
Functional Unit Power	Power contribution from analog and digital functional units on the analog NPU. This includes the consumption of analog and digital compute elements used for neural-network execution, as well as control flow and operation scheduling overhead.
Interconnect Power	Power contribution from the interconnect fabric on the analog NPU, including energy associated with data transfers to and from SRAM, and data movement between compute elements within and across ACE Tiles.
Digital NPU Estimated Power	Total estimated power of the digital NPU using the specified number of digital NPU cores when running the digital portion of the ONNX file.
Total Combined (Analog + Digital NPU) Power	Total combined power of the analog NPU and digital NPU running the compiled ONNX file.

8 PPA Estimator Enhancements

The accuracy of the PPA Estimator is expected to improve as the RTL and circuit implementations of the Mythic GEN 2 hardware continue to mature. As these designs stabilize, additional architectural, RTL, and circuit-level details are incorporated into the estimator, enabling more accurate performance, power, and area (PPA) estimates.

This release includes several enhancements to the PPA Estimator based on the current status of the GEN 2 RTL and circuit development efforts.

8.1 Performance Estimation Updates

The performance estimation flow has been updated to account for the digital NPU used to execute digital nodes within the ONNX graph. These nodes are processed on the Videantis v-MP cores, and the estimator now includes the associated digital processing latency when reporting overall throughput. Reported frame rates reflect the combined execution of the ONNX graph across both the analog NPU and the digital NPU.

8.2 Analog Power Estimation Updates

The analog power estimation flow has been updated to reflect recent enhancements and feature additions in the analog compute architecture. Power estimates for selected analog circuit components have been revised based on the latest SPICE simulation results.

8.3 Digital Power Estimation Updates

The digital power estimation flow has been updated to account for execution of digital ONNX graph nodes on the digital NPU. Reported power estimates reflect the expected power consumption of the Videantis v-MP cores when processing digital nodes at the specified frame rate in addition to the power consumed by the analog NPU.